

# CS T51 – OPERATING SYSTEMS

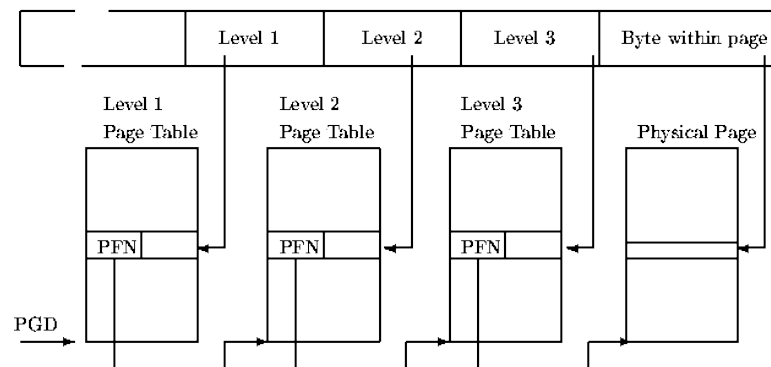
## UNIT – V

File System Structure – File System Implementation – Directory Implementation – Allocation Methods – Free-space Management. Kernel I/O Subsystems - Disk Structure – Disk Scheduling – Disk Management – Swap-Space Management. Case Study: The Linux System, Windows.

### 2 Marks

#### 1. Write the three levels of page tables in Linux systems [Apr 2014]

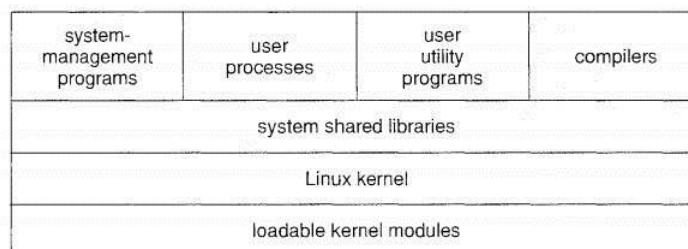
In a three-level system, the first level is only one physical frame of memory. This maps to a second level, which is again only a single frame of memory, and again with the third. Consequently, the three-level system reduces the number of pages required to only a fraction of those required for the single level system.



#### 2. What is meant by Prefetching? [Apr 2014]

Prefetching is the loading of a resource before it is required to decrease the time waiting for that resource. Examples include instruction prefetching where a CPU caches data and instruction blocks before they are executed, or a web browser requesting copies of commonly accessed web pages. Prefetching functions often make use of a cache.

#### 3. Sketch the components of a Linux system (NOV'14)



**4. Define Kernel in OS? (APR'15) (NOV '15)**

Kernel is central component of an operating system that manages operations of computer and hardware. It basically manages operations of memory and CPU time. It is core component of an operating system. Kernel acts as a bridge between applications and data processing performed at hardware level using inter-process communication and system calls.

Kernel loads first into memory when an operating system is loaded and remains into memory until operating system is shut down again. It is responsible for various tasks such as disk management, task management, and memory management.

**5. What type of security is provided by Windows XP (APR'15)?**

- Privileges: Windows XP provides built-in security to keep intruders out by limiting anyone trying to gain access to computer from a network to "guest" level privileges.
- Firewall: As applications and services attempt to send information from a user's computer over the network for the first time, the firewall intercepts the information and lets the user choose whether to allow that communication.

**6. Define physical record and logical record. (NOV '15)**

A physical record often is unstructured and has a fixed size related to the kind of physical media that stores it, and possibly to the location of the record on the media.

A logical record often is structured (has various program-specific fields) and might be stored in some number of full or partial physical records.

For example, a hard disk drive might be divided up into sectors (physical records) of 512 bytes each. If an application needs 1200 bytes of data for an address book entry (logical record) this will occupy portions of 3-4 sectors on disk.

**7. What is the use of boot block? (Apr 16) What is a Boot Control Block? [May 2019]**

For a computer to start running when powered up or rebooted it needs to have an initial program to run. This bootstrap program tends to be simple. It finds the operating system on the disk loads that kernel into memory and jumps to an initial address to begin the operating system execution

Full bootstrap program is stored in the boot blocks at a fixed location on the disk. A disk which has a boot partition is called a boot disk.

**8. What are the functions of Virtual File System (VFS)? (Apr 16)**

- A virtual file system (VFS) is programming that forms an interface between an operating system's Kernel and a more concrete file system
- It manages the data storage and retrieval between the operating system and the storage sub-system
- It gives applications access to different types of file systems and local and network storage devices.
- It separates file system generic operations from their implementation by defining a clean VFS interface. It is also based on a file-representation structure known as vnode, which contains a numerical designator needed to support network file systems.

**9. What is swapping? What is the significant of swap space management? (Nov 16) How do you manage swap space [Nov 2018]What is the use of a swap space? [Nov 2017]State the swap space management technique [Nov 2019]**

Swapping is a memory management technique used in multi-programming to increase the number of process sharing the CPU.

- It is a technique of removing a process from main memory and storing it into secondary memory, and then bringing it back into main memory for continued execution.
- This action of moving a process out from main memory to secondary memory is called Swap Out and the action of moving a process out from secondary memory to main memory is called Swap In.

Swap-Space :The area on the disk where the swapped out processes are stored is called swap space.

Swap-Space Management:

- Swap-space management is another low-level task of the operating system.
- Virtual memory uses disk space as an extension of main memory. Since disk access is much slower than memory access, using swap space significantly decreases system performance.
- The main goal for the design and implementation of swap space is to provide the best throughput for the virtual memory system.

**10. Define seek time and latency time [Apr 2017]**

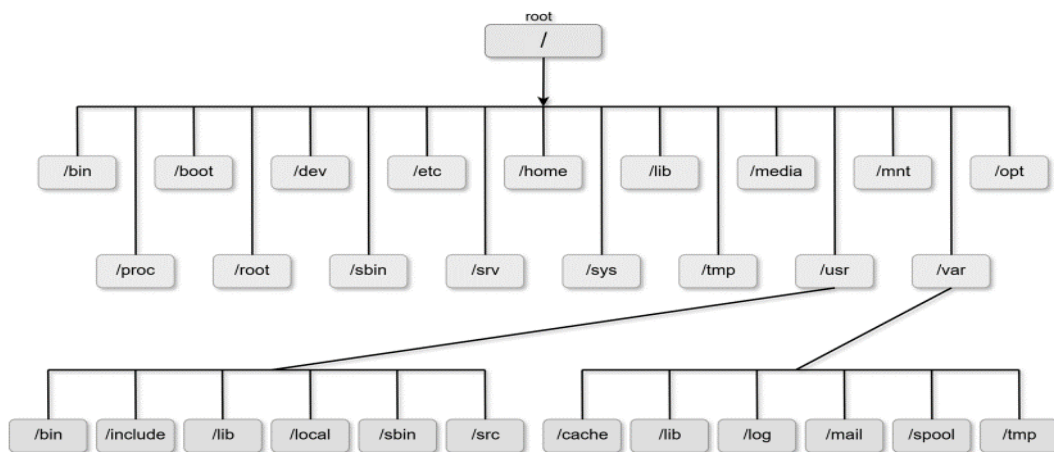
- Seek time is the time required to move the disk arm to the required track.
- Rotational delay or latency is the time it takes for the beginning of the required sector to reach the head.
- Sum of seek time (if any) and latency is the access time. Time taken to actually transfer a span of data is transfer time.

**11. What are the advantages of linked allocation? [Apr 2017]**

- There is no external fragmentation with linked allocation.
- Any free block can be utilized in order to satisfy the file block requests.
- File can continue to grow as long as the free blocks are available.
- Directory entry will only contain the starting block address
- 

**12. What is Disk Management? [Nov 2017]**

**Disk Management** is a utility built into different **operating systems** which can be used to create, delete, format partitions, assign drive letters, and much more. **Disk Management** can also be used to view partitions and their formatted file systems on the **hard drive**

**13. Draw the Linux OS directory structure [Nov 2018]****14. List out the four main object types defined by the Linux VFS [May 2019]**

**superblock, dentries, inodes, files**

**15. Name 4 registers of I/O port [Nov 2019]**

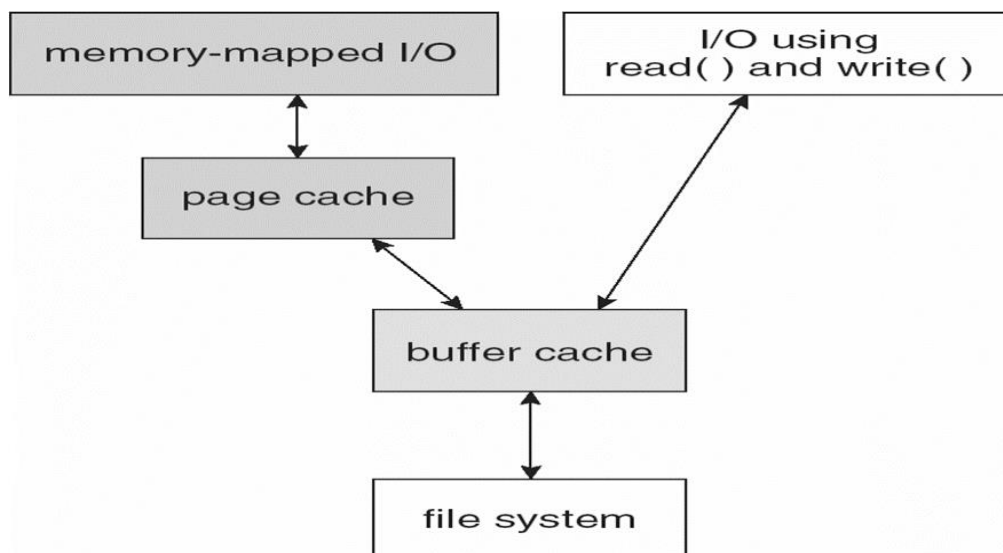
(1) status, (2) control, (3) data-in, and (4) data-out registers.

**11 Marks:****1. Discuss the various techniques used to improve the efficiency and performance of secondary storage? (APR '14)****Efficiency dependent on:**

- Disk allocation and directory algorithms
- Types of data kept in file's directory entry
- Pre-allocation or as-needed allocation of metadata structures
- Fixed-size or varying-size data structures

In a Performance is Keeping data and metadata close together and Buffer cache – separate section of main memory for frequently used blocks ,Synchronous writes sometimes requested by apps or needed by OS, No buffering / caching – writes must hit disk before acknowledgement ,Asynchronous writes more common, buffer-able, faster Free-behind and read-ahead – techniques to optimize sequential access ,Reads frequently slower than writes.

A **page cache** caches pages rather than disk blocks using virtual memory techniques and addresses. Memory-mapped I/O uses a page cache. Routine I/O through the file system uses the buffer (disk) cache. This leads to the following figure



In the above figure A **unified buffer cache** uses the same page cache to cache both memory-mapped pages and ordinary file system I/O to avoid **double caching**. **Consistency checking** is used to compares data in directory structure with data blocks on disk, and tries to fix inconsistencies. Can be slow and sometimes fails. Use system programs to **back up** data from disk to another storage device (magnetic tape, other magnetic disk, optical). Recover lost file or disk by **restoring** data from backup.

## 2. Discuss free space management and allocation methods in file systems [Nov 2017]

Illustrate the implementation of various allocation methods with necessary diagrams and examples [ Apr 2017] [Sep 2020] (APR'15) (Apr 16)(Nov 16)(Apr 17)

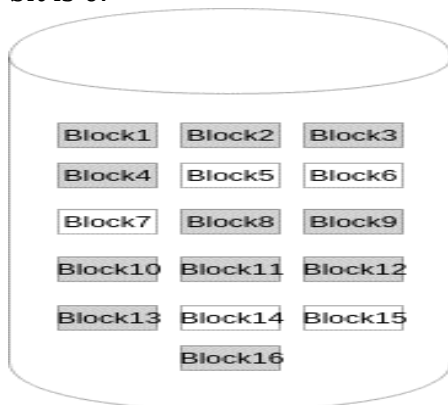
**Free space management :** Since disk space is limited, we need to reuse the space from deleted files for new files , if possible. (Write-once optical disks only allow one write to any given sector, and thus such reuse is not physically possible.) To keep track of free disk space, the system maintains a free-space list. The free-space list records all *free* disk blocks-those not allocated to some file or directory.

### 4 ways to implement free space:

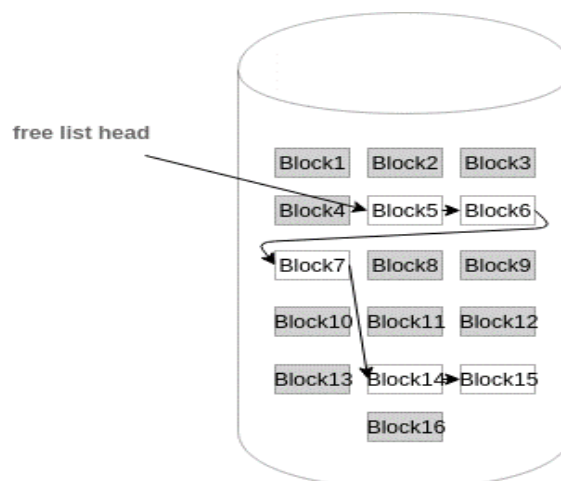
- Bit vector
- Linked List
- Grouping
- Counting

#### Bit vector

Frequently, the free-space list is implemented as a bit vector or bit map. Each block is represented by 1 bit. If the block is free, the bit is 1; if the block is allocated, the bit is 0.



**Linked List** Another approach to free-space management is to link together all the free disk blocks, keeping a pointer to the first free block in a special location on the disk and caching it in memory. This first block contains a pointer to the next free disk block, and so on.



## Grouping

A modification of the free-list approach stores the addresses of  $n$  free blocks in the first free block. The first  $n-1$  of these blocks are actually free. The last block contains the addresses of another  $n$  free blocks, and so on. The addresses of a large number of free blocks can now be found quickly, unlike the situation when the standard linked-list approach is used.

## Counting

Rather than keeping a list of  $n$  free disk addresses, we can keep the address of the first free block and the number ( $n$ ) of free contiguous blocks that follow the first block. Each entry in the free-space list then consists of a disk address and a count. Although each entry requires more space than would a simple disk address, the overall list is shorter, as long as the count is generally greater than 1.

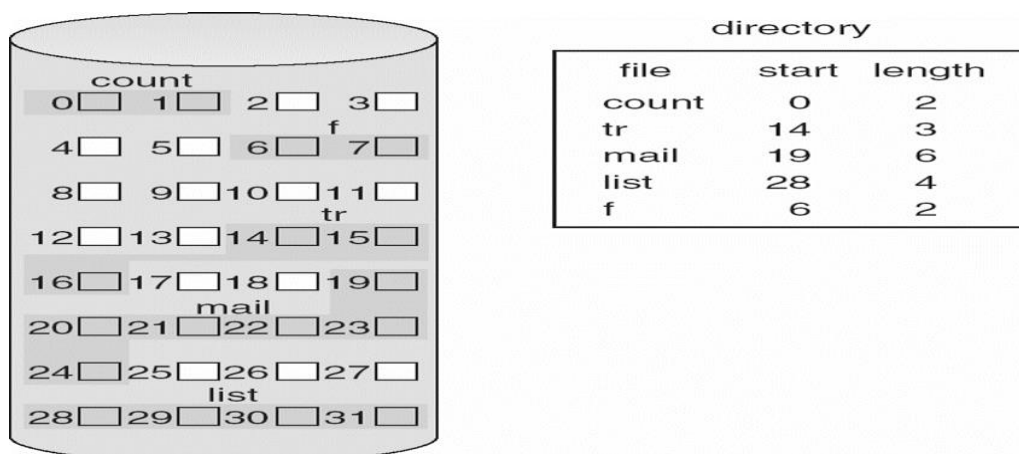
## Allocation methods:

Three major methods of allocating disk space are

1. Contiguous allocation.
2. linked allocation.
3. indexed allocation.

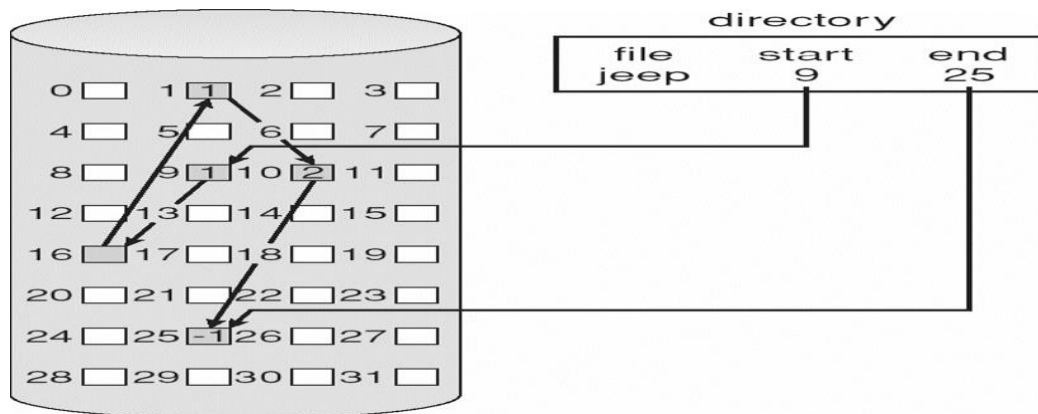
## Contiguous allocation

- A single contiguous set of blocks is allocated to a file at the time of file creation; this is pre allocation strategy that uses portions of variable size. The FAT (File Allocation Table) needs just a single entry for each file, showing the starting block and the length of the file. Disk addresses define a linear ordering on the disk.
- It suffers from external fragmentation. Compaction is used to solve the problem of external fragmentation. The original disk uses then freed completely by creating one large contiguous free space.
- Second problem with contiguous allocation algorithm is that with pre allocation. It is necessary to declare the size of the file at the time of creation.



**Disadvantage:**

- Dynamic allocation is not possible.
- The contiguous disk space allocation problem can be seen to be a particular application of the general dynamic storage allocation.
- Any management system can be used but some are slower than others.
- The other problem is external fragmentation because when it exist free space is broken into chunks.

**Linked allocation:**

- Linked allocation solves all problems of contiguous allocation.
- Each block contains a pointer to the next block in the chain. The disk blocks may be scattered anywhere on the disk, The FAT contains a pointer to the first and last blocks of the file.
- To create a new file, simply create a new entry in FAT .With linked allocation each directory entry has a pointer to the first disk block of the file.
- This pointer is initialized to nil to signify an empty file. Size field is also set to zero. There is no external fragmentation.
- The size of the file does not need to be declared when that file is created. A file can continue to grow as long as free blocks are available. It is never necessary to compact disk space.

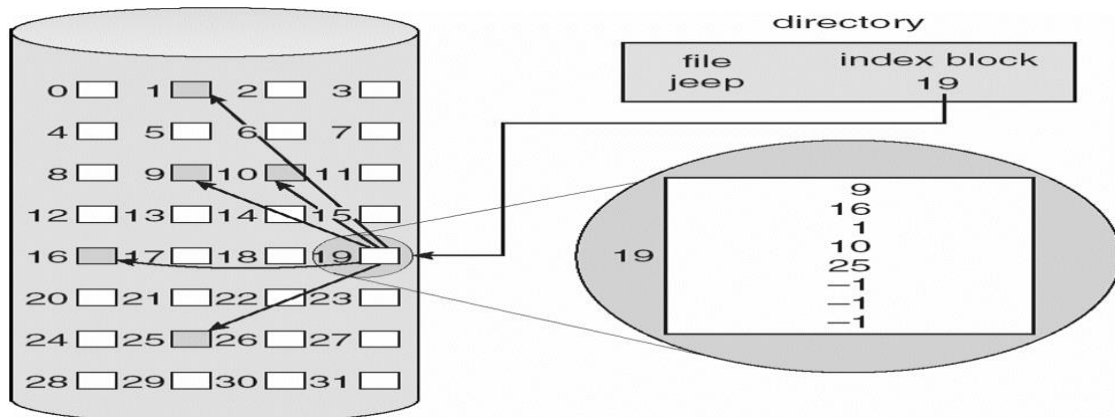
**Disadvantage:**

The major problem is that it can be used effectively only for sequential access files. To find the  $i^{\text{th}}$  block of a file, we must start at the beginning of that file and follow the pointers until we get to the  $i^{\text{th}}$  block.

- Each access to a pointer requires a disk read and sometimes a disk seek
- Space is required for pointers.
- Pointers are used much smaller percentage of the file disk space.
- Other problem in linked allocation is reliability since the files are linked together by pointer scattered all over the disk.
- An important variation on linked allocation method is the use of a file allocation table(FAT).

- FAT is simple but efficient method of disk space allocation is used by MS-DOS and OS /2 operating systems.
- FAT is used same way as linked list.
- FAT allocation scheme can result in a significant number of disk head seeks, unless the FAT is cached.
- The disk head must move to the start of the volume to read the FAT.

### Indexed allocation:



- The FAT contains a separate one level index for each file, the index has one entry for each portion allocated to the file.
- The  $i$ th entry in the index block points to the  $i$ th block of the file.
- The directory contains the address of the index block.
- Typically the file indexes are not physically stored as part of the FAT. Rather the file index for a file is kept in a separate block and the entry for the file in the FAT points to that block.
- Allocation may be on the basis of either fixed size block or variable size portions.
- When the file is created; all pointers in the index block are set to nil.
- When  $i$ th block is first written, a block is obtained from the free space manager and its address is put in the  $i$ th index block entry.
- Allocation by blocks eliminates external fragmentation, whereas allocation by variable size portions improves locality.
- Indexed allocation supports direct access and sequential access to the file and it is the most popular form of file allocation.
- Indexed allocation does not suffer from wasted space. The pointer overhead of the index block is generally greater than the pointer overhead of linked allocation.

### 3. Explain in Detail about Process management? (NOV'14)

UNIX process management separates the creation of processes and the running of a new program into two distinct operations.

1. The fork system call creates a new process
2. A new program is run after a call to execute

Under UNIX, a process encompasses all the information that the operating system must maintain to track the context of a single execution of a single program. Under

Linux, process properties fall into three groups: the process's identity, environment, and context Process Identity

### Process ID (PID)-

The unique identifier for the process; used to specify processes to the operating system when an application makes a system call to signal, modify, or wait for another process

### Credentials-

Each process must have an associated user ID and one or more group IDs that determine the process's rights to access system resources and files

### Personality-

Not traditionally found on UNIX systems, but under Linux each process has an associated personality identifier that can slightly modify the semantics of certain system calls

Used primarily by emulation libraries to request that system calls be compatible with certain specific flavors of UNIX

### Process Environment

The process's environment is inherited from its parent, and is composed of two null-terminated vectors:

1. The argument vector lists the command-line arguments used to invoke the running program; conventionally starts with the name of the program itself
2. The environment vector is a list of "NAME=VALUE" pairs that associates named environment variables with arbitrary textual values

Passing environment variables among processes and inheriting variables by a process's children are flexible means of passing information to components of the user-mode system software. The environment-variable mechanism provides a customization of the operating system that can be set on a per-process basis, rather than being configured for the system as a whole

### Process Context

- The (constantly changing) state of a running program at any point in time
- The **scheduling context** is the most important part of the process context; it is the information that the scheduler needs to suspend and restart the process.
- The kernel maintains **accounting** information about the resources currently being consumed by each process, and the total resources consumed by the process in its lifetime so far.
- The **file table** is an array of pointers to kernel file structures. When making file I/O system calls, processes refer to files by their index into this table .Whereas the file table lists the existing open files, the **file-system context** applies to requests to open new files. The current root and default directories to be used for new file searches are stored here.
- The **signal-handler table** defines the routine in the process's address space to be called when specific signals arrive. The **virtual-memory context** of a process describes the full contents of the private address space.

## Processes and Threads

Linux uses the same internal representation for processes and threads; a thread is simply a new process that happens to share the same address space as its parent. A distinction is only made when a new thread is created by the **clone** system call.

- **fork** creates a new process with its own entirely new process context
- **Clone** creates a new process with its own identity, but that is allowed to share the data structures of its parent.

Using **clone** gives an application fine-grained control over exactly what is shared between two threads.

## 4. Explain in detail about Security? (NOV'14)

### Authentication:

- Authentication in UNIX has typically been performed through the use of a publicly readable password file. A user's password is combined with a random salt value and the result is encoded with a one way transformations functions and stored in the password file.
- The use of the one way function means that the original password cannot be deduced from the password file except by trial and error.
- When a user presents a password to the system the password is recombined with the salt value stored in the password file and passed through the same one way transformations.
- If the result matches the contents of the password file then the password is accepted.
- UNIX implementations of this mechanism have had several problems. Passwords were often limited to eight characters, and the number of possible salt values was so low that an attacker could easily combine a dictionary of commonly used passwords with every possible salt value.
- This will give a good chance of matching one or more passwords in the password file, gaining unauthorized access to any accounts compromised as a result.
- Extensions to the password mechanisms have been introduced that the encrypted password secret in a file that is not publicly readable, that allow longer password, or that use more secure, methods of encoding the password.
- A new security mechanism has been developed by UNIX vendors to address these issues. The Pluggable Authentication Modules (PAM) system.
- It is based on a shared library that can be used by any system components that needs to authenticate users. An implementation of this system is available under Linux.
- PAM allows Authentication modules to be loaded on demand as specified in a system wide configuration file.
- If a new Authentication mechanism is added at a later date, it can be added to the configuration file and all system components will immediately be able to take advantage of it.
- PAM modules can specify Authentication methods, account restrictions, session set up functions, or password changing functions (so that when users change their password, all the necessary Authentication mechanisms can be updated

at once).

**Access Control:**

- Access control under UNIX systems, including Linux, is performed through the use of unique numeric identifiers.
- A user identifier (uid) identifies a single user or a single set of access rights. A group identifier (gid) is an extra identifier that can be used to identify rights belonging to more than one user.
- Access control is applied to various objects in the system. Every file available in the system is protected by a standard access control mechanism.
- In addition, other shared objects, such as shared-memory sections and semaphores, employ the same access systems.
- Every object in a UNIX system under and group access control has a single uid and a single gid associated with it. User processes also have a single uid, but they may have more than one gid.
- If a process's uid matches the uid of an object, then the process the user rights or owner rights to that object. If the uids do not match but any of the process' gids match the object's gid, then group rights are conferred; otherwise, the process has world rights to the object.
- Linux performs access control by assigning objects a protection mask that specifies which access modes-read, write or execute-are to be granted to process with owner, group, or world access.
- Thus, the owner of an object might have full read, write and execute access to a file; other users in a certain group might be given read access but denied write access; and everybody else might be given no access at all.
- The only exception is the privileged root uid. A process with this special uid is granted to any object in the system, bypassing normal access checks.
- Such processes are also granted permission to perform privileged operations such as reading any physical memory or opening reserved network sockets.
- This mechanism allows the kernel to prevent normal users from accessing these resources. Most of the kernel's key internal resources are implicitly owned, by this root uid.

**5.Discuss the disk scheduling algorithms for storage management [Nov 2017]**

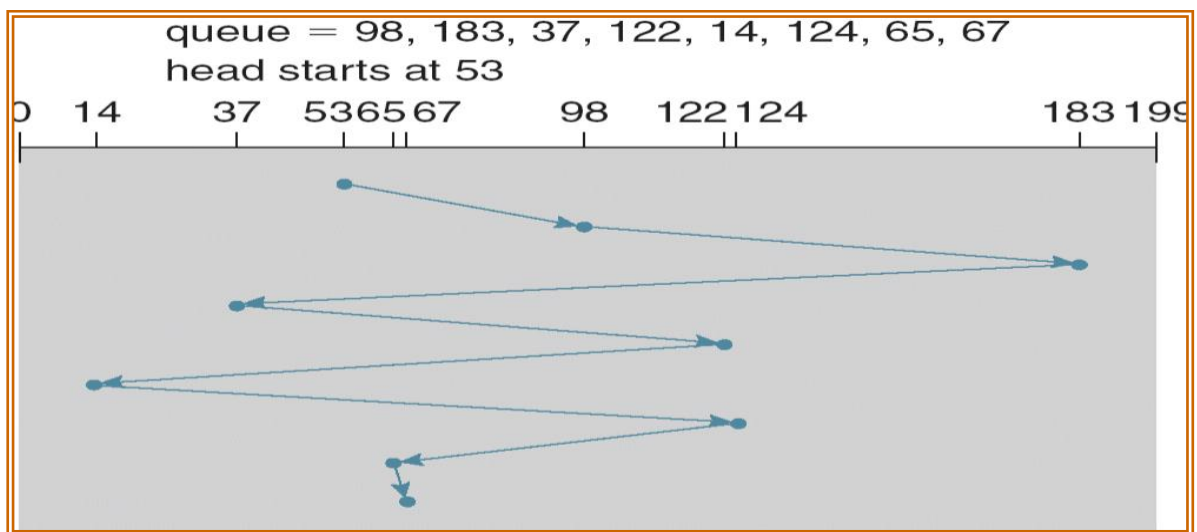
Disk Scheduling the amount of head movement needs to satisfy a series of I/O request can affect the performance. If the desired disk drive and controller are available; the request can be serviced immediately. If a device or controller is busy; any new requests for service will be placed on the queue of pending requests for that drive. When one request is completed; the OS chooses which pending request to service next.

**Different types of scheduling algorithm are as follows:**

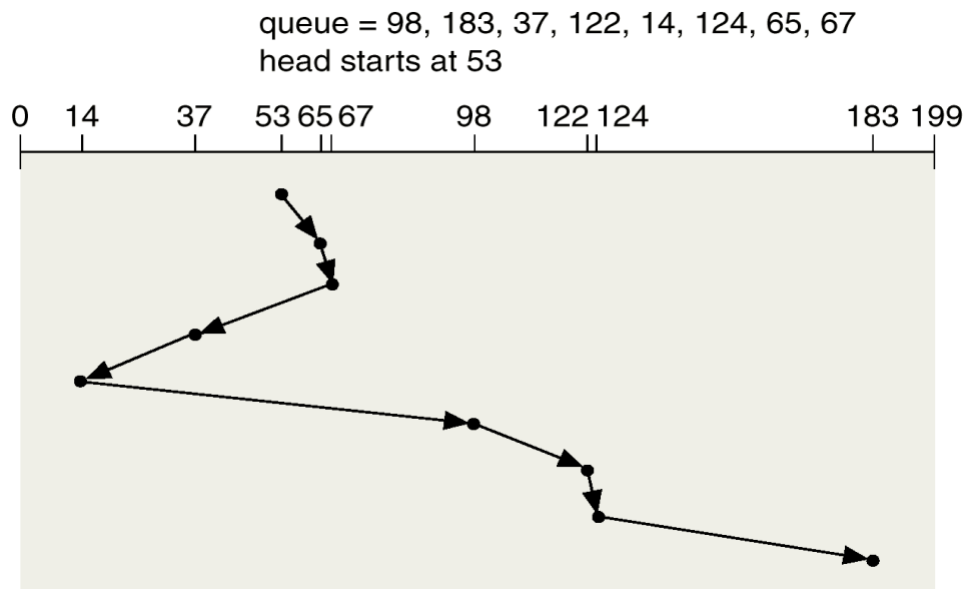
1. First come first serviced scheduling algorithm
2. Shortest seek time first algorithm
3. SCAN
4. Seek (or LOOK)
5. Circular scans(C-SCAN)
6. C-Seek (or C-LOOK)

**FCFS Scheduling:**

- FCFS-FIRST COME FIRST SERVE is the simplest form of disk scheduling.
- Service requests in the order they are received. This algorithm is similar to the FCFS job scheduling algorithm.
- FCFS is easy to implement but does not guarantee good throughput.
- It does not provide the fastest service.
- It takes no special action to minimize the overall seek time.
- This can be explained with an example as follows:
- Consider disk queue with requests for I/O to blocks in cylinders are, 98, 183, 37, 122, 14, 124, 65, 67.
- Here if the disk head is initially at 53, then it will move from 53 to 98 and then to 183. This happens for a total head movements of 640 cylinders.
- Here the total head movements can be decreased simultaneously. Suppose for cylinders 37 & 14 has to be serviced together, before or after the requests at 122 & 124, then the head movements can be decreased.

**SSTF Scheduling:**

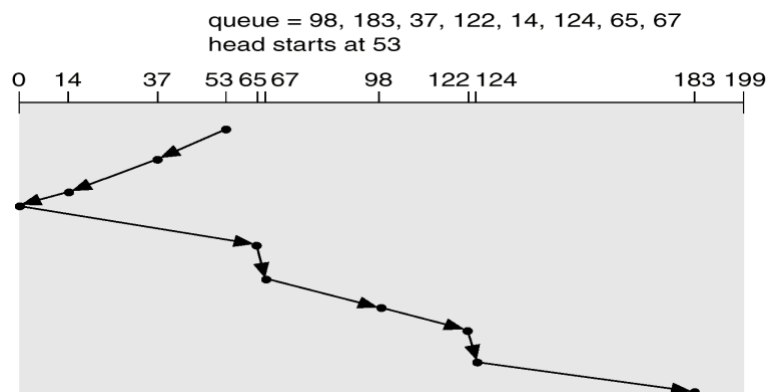
- This algorithm selects the request with the minimum seek time from the current head position.
- SSTF may tend to move the head away from some requests during its local minimization.
- Seek time increases with the number of cylinders traversed by the head, SSTF chooses the pending request closest to the current head position so always choosing the minimum seek time does not guarantee that the average seek time over a number of arm movements will be minimum. This choice should provide better performance than FCFS algorithm.



- 
- From the above FCFS example, the closest request to the initial head position 53 is at cylinder 65. once we are at cylinder 65, the next closest request is at cylinder 67. from these, the request at cylinder 37 is closer than 98, so 37 is served next then finally we will reach 183.
- This scheduling method results in a total head movement of only 236 cylinders little more than one-third of the distance needed for FCFS scheduling of this request queue.

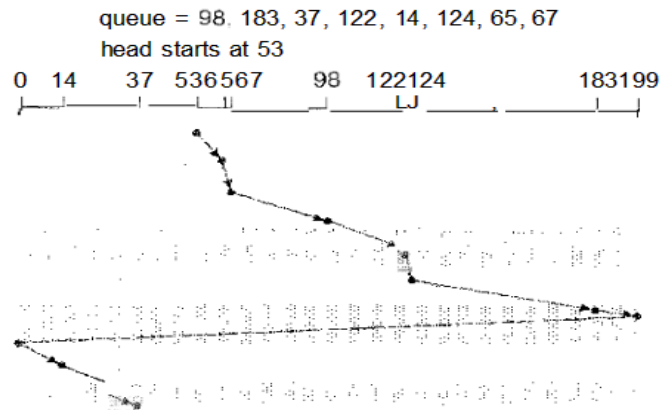
### Scan Scheduling:

- In this type of algorithm, the disk can start at one end of the disk and moves towards the other end.
- At the other end the direction of head movement is reversed and servicing continues.
- This algorithm is also otherwise called as elevator algorithm. before applying this algorithm, we should know the direction of head movement, in addition to the head current position.
- If the disk arm is moving towards 0, then the head will move towards the other end of the disk servicing the request at 65, 67, 98, 122, 124 & 183. first we have to service all
- The request going up, and then reversing it.



### C-Scan Scheduling:

- C-SCAN SCHEDULING (CIRCULAR SCAN) is a variant of scan designed to provide a more uniform wait time.
  - When the head reaches the other end, however it immediately returns to the beginning of the disk, without servicing any request on the return trip.
  - It essentially treats the cylinder as a circular list.

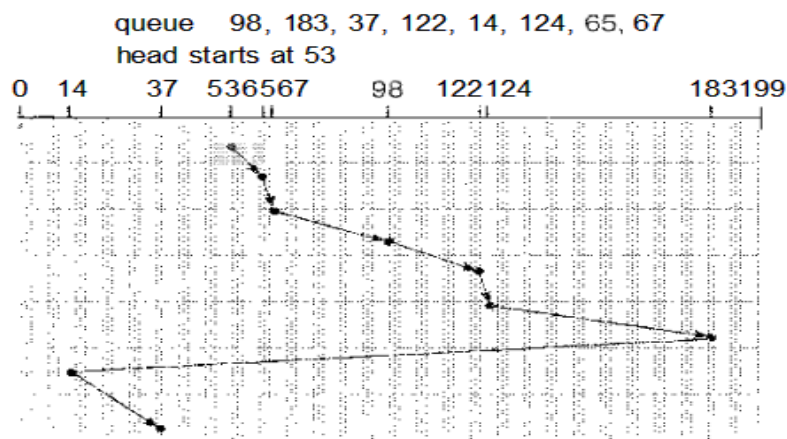


### LOOK scheduling:

- The versions of C-SCAN and SCAN scheduling algorithm are called C-LOOK and LOOK scheduling.
- Both the algorithms move the disk across its full width. This is because they look for a request before continuing to move in a given direction.

### C-LOOK scheduling:

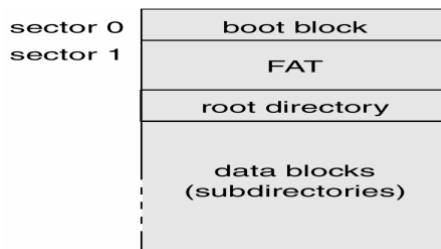
- This is a modification of seek algorithm, similar to C-SCAN. Here the arm goes only as far as the requests to be serviced and requests are serviced only in one direction.
- It is the best algorithm, since it involves minimum movement of the disk head to service all the requests. Disk head will not move, if there is no pending requests thus reducing wear and tear of the disk.



## 6. Briefly explain Disk management and swap space management? (Apr 16)

Low level formatting or physical formatting: Dividing a disk into sectors that the disk controller can read and write. To use a disk to hold files, the operating system still needs to record its own data structures on the disk. Partition the disk into one or more groups of cylinders. Logical formatting or “making a file system”. Boot block initializes system. The bootstrap is stored in ROM. Bootstrap loader program. Methods such as sector sparing used to handle bad blocks.

### MS-DOS Disk Layout



Methods such as sector sparing (also known as forwarding) used to handle bad blocks. Spare sectors set aside on low-level formatting – Controller told to replace a bad sector logically with one of the spare sectors – To retain effectiveness of disk-scheduling optimization, provide spare sectors in each cylinder and also provides some spare cylinders. Use spare sector from same cylinder if possible.

Sector slipping is moves blocks following bad block downward (occupying spare sector) to free up block following bad Block skips bad block, using freed up block to hold that sector's information.

### Swap-Space Management

Swap space Virtual memory uses disk space as an extension of main memory. Swap-space can be carved out of the normal file system, or, more commonly, it can be in a separate disk partition. Swap-space management allocates swap space 4.3BSD. When process starts, holds text segment (the program) and data segment. Kernel uses swap maps to track swap-space use. Solaris 2 allocates swap space only when a page is forced out of physical memory, not when the virtual memory page is first created. File data written to swap space until write to file system requested other dirty pages go to swap space due to no other home. Text segment pages thrown out and reread from the file system as needed

## 7. Write notes on Linux kernel architecture. Advantages of Linux over windows OS. [Nov 2018]

Linux is a modern, free operating system based on UNIX standards. First developed as a small but self-contained kernel in 1991 by Linux Torvalds, with the major design goal of UNIX compatibility. Its history has been one of collaboration by many users from all around the world, corresponding almost exclusively over the Internet. It has been designed to run efficiently and reliably on common PC hardware,

but also runs on a variety of other platforms. The core Linux operating system kernel is entirely original, but it can run much existing free.

### **The Linux Kernel**

Version 0.01 (May 1991) had no networking, ran only on 80386-compatible Intel processors and on PC hardware, had extremely limited device-driver support, and supported only the Minix file system

### **Linux Distributions:**

Standard, precompiled sets of packages, or *distributions*, include the basic Linux system, system installation and management utilities, and ready-to-install packages of common UNIX tools. The first distributions managed these packages by simply providing a means of unpacking all the files into the appropriate places; modern distributions include advanced package management. Early distributions included SLS and **Slack ware**. **Slack ware** represents overall improvement in quality. **Red Hat and Debian** are popular distributions from commercial and noncommercial sources, respectively

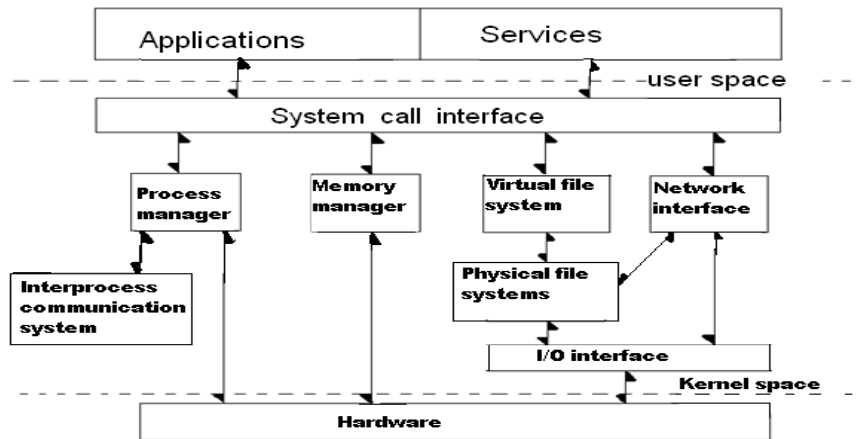
The RPM Package file format permits compatibility among the various Linux distributions

### **Linux Licensing**

- The Linux kernel is distributed under the GNU General Public License (GPL), the terms of which are set out by the Free Software Foundation
- Anyone using Linux, or creating their own derivative of Linux, may not make the derived product proprietary; software released under the GPL may not be redistributed as a binary-only product

### **Design Principles**

1. Linux is a multi-user, multitasking system with a full set of UNIX-compatible tools
2. Its file system adheres to traditional UNIX semantics, and it fully implements the standard UNIX networking model
3. Main design goals are speed, efficiency, and standardization
4. Linux is designed to be compliant with the relevant POSIX documents; at least two Linux distributions have achieved official POSIX certification
5. The Linux programming interface adheres to the SVR4 UNIX semantics, rather than to BSD behaviour



## COMPONENTS OF A LINUX SYSTEM

Like most UNIX implementations, Linux is composed of three main bodies of code; the most important distinction between the kernel and all other components. The **kernel** is responsible for maintaining the important abstractions of the operating system. Kernel code executes in *kernel mode* with full access to all the physical resources of the computer. All kernel code and data structures are kept in the same single address space. The **system libraries** define a standard set of functions through which applications interact with the kernel, and which implement much of the operating-system functionality that does not need the full privileges of kernel code. The **system utilities** perform individual specialized management tasks. **Components of a Linux system shown below**

|                            |                |                       |           |
|----------------------------|----------------|-----------------------|-----------|
| System management programs | User processes | User utility programs | compilers |
| System shared libraries    |                |                       |           |
| Linux kernel               |                |                       |           |
| Loadable kernel modules    |                |                       |           |

### Kernel Modules

Sections of kernel code that can be compiled, loaded, and unloaded independent of the rest of the kernel

- A kernel module may typically implement a device driver, a file system, or a networking protocol
- The module interface allows third parties to write and distribute, on their own terms, device drivers or file systems that could not be distributed under the GPL

- Kernel modules allow a Linux system to be set up with a standard, minimal kernel, without any extra device drivers built in

Three components to Linux module support:

- **Module management** –allows modules to be loaded into memory.
- **Driver registration**–allows modules to tell the rest of the kernel that a new driver has become available.
- **Conflict resolution**–allows different device drivers to reserve hardware resources and to protect those resources from accidental use by another driver.

### Module Management

Supports loading modules into memory and letting them talk to the rest of the kernel.

Module loading is split into two separate sections:

1. Managing sections of module code in kernel memory
2. Handling symbols that modules are allowed to reference

The module requestor manages loading requested, but currently unloaded, modules; it also regularly queries the kernel to see whether a dynamically loaded module is still in use, and will unload it when it is no longer actively needed

### Driver Registration

Allows modules to tell the rest of the kernel that a new driver has become available the kernel maintains dynamic tables of all known drivers, and provides a set of routines to allow drivers to be added to or removed from these tables at any time. Registration tables include the following items:

### Device drivers-

These drivers include character devices (such as printers, terminals) block devices (including all disk drivers ), and network interface devices.

### File systems –

It implements Linux's virtual –file –system calling routines.

### Network protocols-

It implements the entire networking protocol, such as IPX, or a new set of packet-filtering rules for a network firewall.

### Binary format-

Specifies a way of recognizing, and loading ,a new type of executable file.

### Conflict Resolution

A mechanism that allows different device drivers to reserve hardware resources and to protect those resources from accidental use by another driver.

The conflict resolution module aims to:

- Prevent modules from clashing over access to hardware resources
- Prevent **auto probes**from interfering with existing device drivers
- Resolve conflicts with multiple drivers trying to access the same hardware

### Explain how interprocess communication is carried out in Linux OS? (APR'15)

Processes communicate with each other and with the kernel to coordinate their activities. Linux supports a number of Inter-Process Communication (IPC) mechanisms. Signals and pipes are two of them but Linux also supports the System V IPC mechanisms named after the Unix release in which they first appeared.

### Signals

Signals are one of the oldest inter-process communication methods used by Unix™ systems. They are used to signal asynchronous events to one or more processes. A signal could be generated by a keyboard interrupt or an error condition such as the process attempting to access a non-existent location in its virtual memory. Signals are also used by the shells to signal job control commands to their child processes.

There are a set of defined signals that the kernel can generate or that can be generated by other processes in the system, provided that they have the correct privileges. You can list a system's set of signals using the kill command (kill -l), on my Intel Linux box this gives:

|               |             |              |             |
|---------------|-------------|--------------|-------------|
| 1) SIGHUP     | 2) SIGINT   | 3) SIGQUIT   | 4) SIGILL   |
| 5) SIGTRAP    | 6) SIGIOT   | 7) SIGBUS    | 8) SIGFPE   |
| 9) SIGKILL    | 10) SIGUSR1 | 11) SIGSEGV  | 12) SIGUSR2 |
| 13) SIGPIPE   | 14) SIGALRM | 15) SIGTERM  | 17) SIGCHLD |
| 18) SIGCONT   | 19) SIGSTOP | 20) SIGTSTP  | 21) SIGTTIN |
| 22) SIGTTOU   | 23) SIGURG  | 24) SIGXCPU  | 25) SIGXFSZ |
| 26) SIGVTALRM | 27) SIGPROF | 28) SIGWINCH | 29) SIGIO   |
| 30) SIGPWR    |             |              |             |

The numbers are different for an Alpha AXP Linux box. Processes can choose to ignore most of the signals that are generated, with two notable exceptions: neither the SIGSTOP signal which causes a process to halt its execution nor the SIGKILL signal which causes a process to exit can be ignored. Otherwise though, a process can choose just how it wants to handle the various signals. Processes can block the signals and, if they do not block them, they can either choose to handle them themselves or allow the kernel to handle them. If the kernel handles the signals, it will do the default actions required for this signal. For example, the default action when a process receives the SIGFPE (floating point exception) signal is to core dump and then exit. Signals have no inherent relative priorities. If two signals are generated for a process at the same time then they may be presented to the process or handled in any order. Also there is no mechanism for handling multiple signals of the same kind. There is no way that a process can tell if it received 1 or 42 SIGCONT signals.

Linux implements signals using information stored in the task\_struct for the process.

The number of supported signals is limited to the word size of the processor. Processes with a word size of 32 bits can have 32 signals whereas 64 bit processors like the Alpha AXP may have up to 64 signals. The currently pending signals are kept in the signal field with a mask of blocked signals held in blocked. With the exception of SIGSTOP and SIGKILL, all signals can be blocked. If a blocked signal is generated, it remains pending until it is unblocked. Linux also holds information about how each

process handles every possible signal and this is held in an array of sigaction data structures pointed at by the task\_struct for each process. Amongst other things it contains either the address of a routine that will handle the signal or a flag which tells Linux that the process either wishes to ignore this signal or let the kernel handle the signal for it. The process modifies the default signal handling by making system calls and these calls alter the sigaction for the appropriate signal as well as the blocked mask.

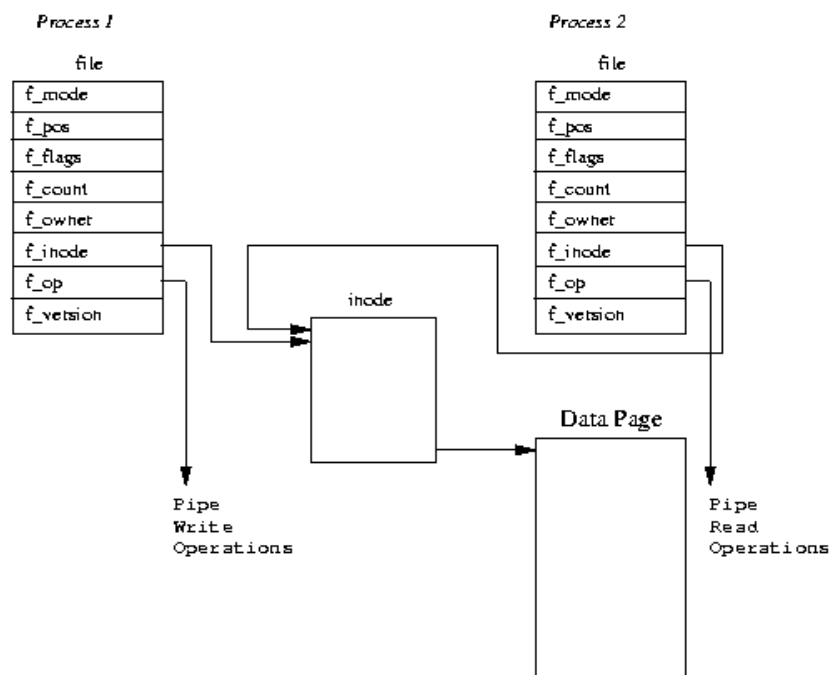
Linux is POSIX compatible and so the process can specify which signals are blocked when a particular signal handling routine is called. This means changing the blocked mask during the call to the processes signal handler. The blocked mask must be returned to its original value when the signal handling routine has finished. Therefore Linux adds a call to a tidy up routine which will restore the original blocked mask onto the call stack of the signalled process. Linux also optimizes the case where several signal handling routines need to be called by stacking them so that each time one handling routine exits, the next one is called until the tidy up routine is called.

## Pipes

The common Linux shells all allow redirection. For example

```
$ ls | pr | lpr
```

pipes the output from the ls command listing the directory's files into the standard input of the pr command which paginates them. Finally the standard output from the pr command is piped into the standard input of the lpr command which prints the results on the default printer. Pipes then are unidirectional byte streams which connect the standard output from one process into the standard input of another process. Neither process is aware of this redirection and behaves just as it would normally. It is the shell which sets up these temporary pipes between the processes.



In Linux, a pipe is implemented using two file data structures which both point at the same temporary VFS inode which itself points at a physical page within memory. Figure shows that each file data structure contains pointers to different file operation routine vectors; one for writing to the pipe, the other for reading from the pipe.

This hides the underlying differences from the generic system calls which read and write to ordinary files. As the writing process writes to the pipe, bytes are copied into the shared data page and when the reading process reads from the pipe, bytes are copied from the shared data page. Linux must synchronize access to the pipe. It must make sure that the reader and the writer of the pipe are in step and to do this it uses locks, wait queues and signals.

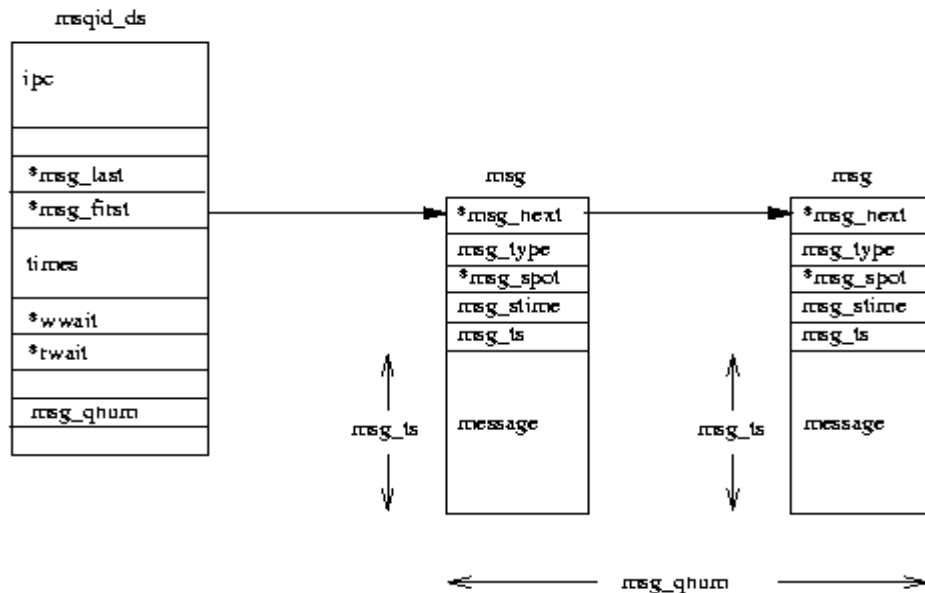
When the writer wants to write to the pipe it uses the standard write library functions. These all pass file descriptors that are indices into the process's set of file data structures, each one representing an open file or, as in this case, an open pipe. The Linux system call uses the write routine pointed at by the file data structure describing this pipe. That write routine uses information held in the VFS inode representing the pipe to manage the write request.

Reading data from the pipe is a very similar process to writing to it.

Linux also supports *named* pipes, also known as FIFOs because pipes operate on a First In, First Out principle. The first data written into the pipe is the first data read from the pipe. Unlike pipes, FIFOs are not temporary objects; they are entities in the file system and can be created using the `mkfifo` command. Processes are free to use a FIFO so long as they have appropriate access rights to it. The way that FIFOs are opened is a little different from pipes. A pipe (its two file data structures, its VFS inode and the shared data page) is created in one go whereas a FIFO already exists and is opened and closed by its users. Linux must handle readers opening the FIFO before writers open it as well as readers reading before any writers have written to it. That aside, FIFOs are handled almost exactly the same way as pipes and they use the same data structures and operations.

## Message Queues

Message queues allow one or more processes to write messages, which will be read by one or more reading processes. Linux maintains a list of message queues, the `msgque` vector; each element of which points to `amsqid_ds` data structure that fully describes the message queue. When message queues are created a new `msqid_ds` data structure is allocated from system memory and inserted into the vector.( Figure (b))



Each `msqid_ds` data structure contains an `ipc_perm` data structure and pointers to the messages entered onto this queue. In addition, Linux keeps queue modification times such as the last time that this queue was written to and so on. The `msqid_ds` also contains two wait queues; one for the writers to the queue and one for the readers of the message queue.

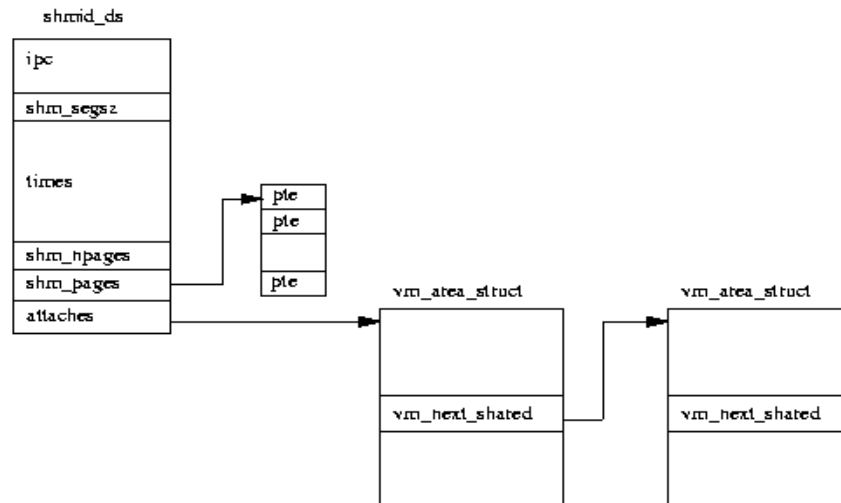
Each time a process attempts to write a message to the write queue its effective user and group identifiers are compared with the mode in this queue's `ipc_perm` data structure. If the process can write to the queue then the message may be copied from the process's address space into a `msg` data structure and put at the end of this message queue. Each message is tagged with an application specific type, agreed between the cooperating processes. However, there may be no room for the message as Linux restricts the number and length of messages that can be written. In this case the process will be added to this message queue's write wait queue and the scheduler will be called to select a new process to run. It will be woken up when one or more messages have been read from this message queue.

Reading from the queue is a similar process. Again, the processes access rights to the write queue are checked. A reading process may choose to either get the first message in the queue regardless of its type or select messages with particular types. If no messages match this criteria the reading process will be added to the message queue's read wait queue and the scheduler run. When a new message is written to the queue this process will be woken up and run again.

## Shared Memory

Shared memory allows one or more processes to communicate via memory that appears in all of their virtual address spaces. The pages of the virtual memory are referenced by page table entries in each of the sharing processes' page tables. It does not have to be at the same address in all of the processes' virtual memory. As with all System V IPC objects, access to shared memory areas is controlled via keys and access rights checking. Once the memory is being shared, there are no checks on how the

processes are using it. They must rely on other mechanisms, for example System V semaphores, to synchronize access to the memory.



Each newly created shared memory area is represented by a `shmid_ds` data structure. These are kept in the `shm_segsz` vector.

The `shmid_ds` data structure describes how big the area of shared memory is, how many processes are using it and information about how that shared memory is mapped into their address spaces. It is the creator of the shared memory that controls the access permissions to that memory and whether its key is public or private. If it has enough access rights it may also lock the shared memory into physical memory. Each process that wishes to share the memory must attach to that virtual memory via a system call. This creates a new `vm_area_struct` data structure describing the shared memory for this process. The process can choose where in its virtual address space the shared memory goes or it can let Linux choose a free area large enough. The new `vm_area_struct` structure is put into the list of `vm_area_struct` pointed at by the `shmid_ds`. The `vm_next_shared` and `vm_prev_shared` pointers are used to link them together. The virtual memory is not actually created during the attach; it happens when the first process attempts to access it.

The first time that a process accesses one of the pages of the shared virtual memory, a page fault will occur. When Linux fixes up that page fault it finds the `vm_area_struct` data structure describing it. This contains pointers to handler routines for this type of shared virtual memory. The shared memory page fault handling code looks in the list of page table entries for this `shmid_ds` to see if one exists for this page of the shared virtual memory. If it does not exist, it will allocate a physical page and create a page table entry for it. As well as going into the current process's page tables, this entry is saved in the `shmid_ds`. This means that when the next process that attempts to access this memory gets a page fault, the shared memory fault handling code will use this newly created physical page for that process too. So, the first process that accesses a page of the shared memory causes it to be created and thereafter access by the other processes cause that page to be added into their virtual address spaces.

When processes no longer wish to share the virtual memory, they detach from it. So long as other processes are still using the memory the detach only affects the current process. Its `vm_area_struct` is removed from the `shmid_ds` data structure and deallocated. The current process's page tables are updated to invalidate the area of virtual memory that it used to share. When the last process sharing the memory detaches from it, the pages of the shared memory current in physical memory are freed, as is the `shmid_ds` data structure for this shared memory.

### **Explain in detail about Windows XP System Architecture? (APR'15)**

The architecture of windows xp consist of 2 modes

- Protected mode
- User mode

The main layers of protected mode are hardware abstraction layer, kernel and executive.

The collection of subsystem and services are called as user mode.

User mode subsystems fall into two categories.

- Environment subsystem – which emulate different operating system
- Protection subsystem – which provide security function

#### **Protected mode:**

##### **1. Hardware abstraction layer :**

The HAL is the layer of software that hides hardware differences from upper levels of the operating system to help make Windows XP portable.

The HAL exports a virtual machine interface that is used by the kernel dispatcher , executive and the drivers.

The advantage of this approach is that only a single version of each device driver is required.

The HAL also provide support for symmetric multiprocessing.

##### **2. Kernel:**

The kernel of windows xp provides the foundations for the executive and the subsystems.

The kernel remains in memory and its execution is never preempted. It has 4 responsibilities.

They are thread scheduling, interrupt and execution handling, low-level processor synchronization and recovery after a power failure.

The kernel is object oriented. Kernel has kernel dispatcher.

Kernel dispatcher:

- Kernel dispatcher provides the foundation for the executive and subsystems.
- The dispatcher is never paged out of memory and its execution is never preempted.
- Its main responsibilities are thread scheduling , implementation of synchronization primitives , software interrupts and execution dispatching.

Thread scheduling:

- Windows xp uses the concepts of processor and thread for executable code.

- The process has a virtual memory address and information used to initialize each thread.
- Each process has one or more thread each of which is an executable unit dispatched by the kernel. There are 6 possible thread states. They are

**Ready** – indicates waiting to run

**Standby** – highest priority ready thread is moved to standby state

**Running** –executing on a processor

**Waiting** –waiting for an dispatcher object

**Transition** –a new thread is in transition state while its waits for resources necessary for execution.

**Termination** –finishes execution.

- Implementation of synchronization primitives:

The os data structures are managed as object using common facilities for allocation, reference counting and security.

- Software interrupt:

2 types of software interrupt

- Asynchronous procedure call:

It is used to begin execution of a new thread, terminate processes and to deliver notification that an asynchronous I/O has completed.

- Deferred procedure call:

It is used to postpone interrupt processing.

- Exception and interrupts:

The Windows XP has several exception including

1. Memory access violation
2. Integer overflow
3. Floating point overflow or underflow
4. Integer divide by zero
5. Floating point divide by zero etc.

This exception are handled by the exception dispatcher when an exception occurs in kernel mode, the exception dispatcher simply calls a routine to locate the exception handler.

### 3. Executive :

The Windows XP executive provides a set of services that all environment subsystems use.

The services are grouped as follows:

- Object manager ,
- virtual memory manager ,
- process manager ,
- local procedure call facility ,
- I/O manager,
- security references monitor ,
- plug and play and security manager ,
- register and booting.

- Object manager

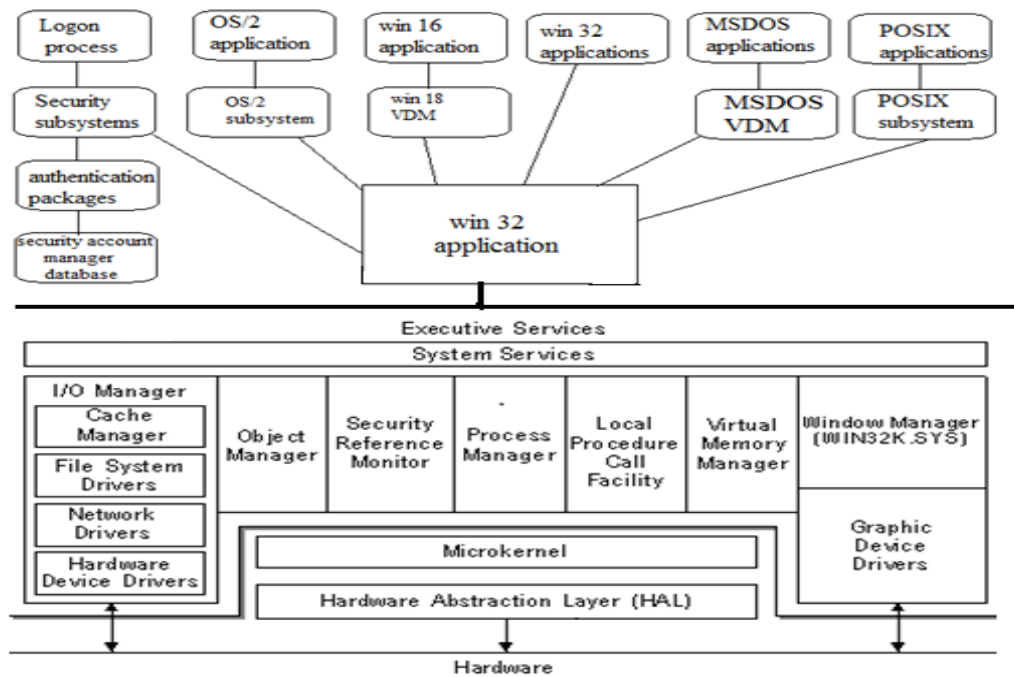
Windows XP uses a generic set of interface s for managing the kernel entities that is manipulating by user mode program. Windows XP calls these entities objects

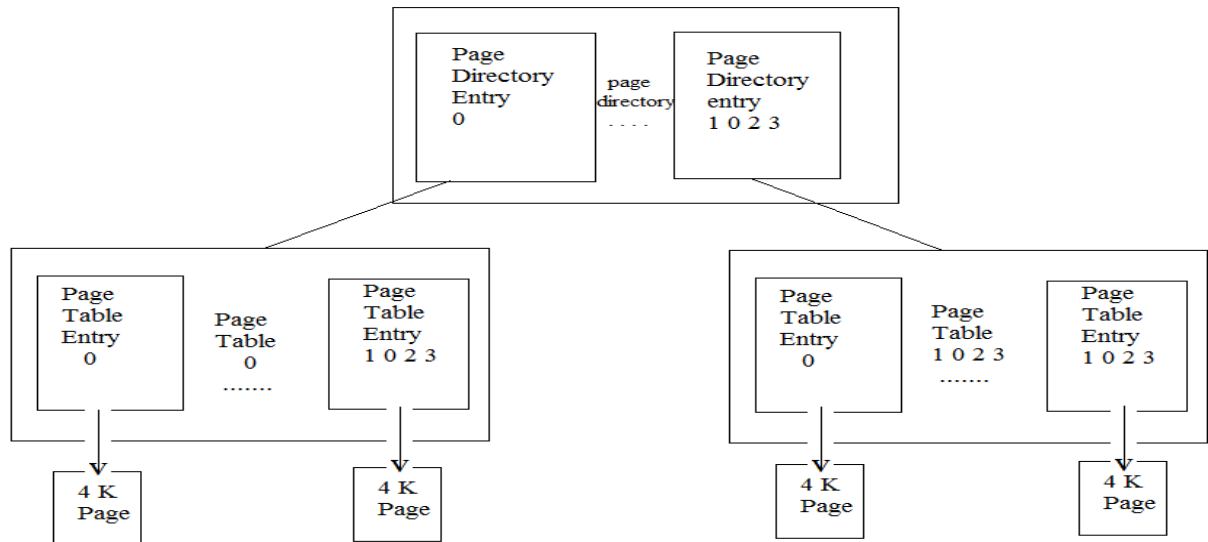
and the executive component that manipulate them is the object manager. The job of the object manager is to supervise the use of all the managed objects. When a thread wants to use an object, it calls the object managers open method to get a reference to the object.

➤ Virtual memory manager:

Then executive component that manages the virtual address space, physical memory allocation and paging is the virtual memory manager. The design of VM manager assumes that the underlying hardware support virtual to physical mapping, a paging mechanism, transparent cache coherence on multiprocessor system and allow multiple page table entries to map to the same physical page frame.

The VM manager for IA32 processor has a page directory that contains 1024 page directory entries (PDE) of size 4 bytes. Each PTE point to a 4 kb page frames.





A 32 bit virtual memory address is splitted into 3 value

- i. First 10 bit – used as an index into page directory
- ii. Next 10 bit – used to select a file from page table
- iii. The remaining 12 bits are the offset of the specific in the page frame.

| PDE | PTE | Page offset |
|-----|-----|-------------|
|-----|-----|-------------|

- Process manager:
  - The windows xp process manager provides services for creating, deleting and using processes, threads and jobs.
  - It has no knowledge about parent child relationship or process hierarchies.
  - The pro manager is also not involved in the scheduling of processes other than setting the priorities
  - And the affirmatives in processes and threads when they are created.
  - Thread scheduling takes place in kernel dispatcher.
- Local procedure call facility:
  - The implementation of Windows XP uses a client server model.
  - The OS uses local procedure call (LPC) to pass request and result between client and server process within a single machine.

### The various Windows XP subsystems.

When an LPC channel is created, one of three message passing techniques must be specified.

1. The first tech is suitable for small messages. In this parts message queue is used as intermediate storage, the message are copied from one process to the other.
2. The second tech is for large messages. Message send through the ports message queue contain a pointer and size information referring to the section object.

This avoids the need to copy large messages. The sender places data into the shared section and receiver views them directly.

3. The third tech of LPC message passing uses the API that read / writes directly into a process address space.

➤ I/O manager :

- I/O manager is responsible for file system, device drivers and network drivers.
- It keeps track of which device drivers, filter drivers and file systems are loaded and also manages buffer for I/O request.
- Device driver are arranged as a list for each the I/O manager convert the request it receive into standard form called as I/O request packet (IRP)

➤ Cache manager:

In many OS, caching is done by the file system. The cache manager works closely with the VM manager to provide cache services for all components under the control of I/O manager.

➤ Security reference monitor:

The security reference monitor (SRM) is also responsible for manipulating the privilege in security tokens.

Whenever a process opens a handle to an object the security reference monitor (SRM) checks the process.

➤ Plug and play and power manager:

The OS uses the Plug and play manager to recognize and adapt the change in the hardware configuration.

Windows XP also moves to system to a state requiring lower power consumption.

## 2. User mode:

### 1. Environmental subsystem:

It is a user modes processes layered over the native windows xp executive services to enable windows xp to run program developed for other os including 16 bit windows , MS DOS and POSIX.

- MS DOS environment:
  - The MS-DOS environment does not have of other windows xp environment subsystems.
  - It is provided by a win 32 application called the virtual DOS machine (VDM)
- 16-bit windows environment:
  - The win16 execution environment is provided by a VDM that incorporates additional software called windows on windows that provides the windows 3.1 kernel routines and stub routine for window manager and graphical device interface functions.
  - POSIX subsystem:
    - The POSIX system is designed to run POSIX application written to follow the POSIX standard, which is based on the UNIX model.
    - POSIX applications can be started by the win 32 sub system or by another POSIX application.

## 2. Logon and security subsystems:

Before a user can access objects on windows xp, that user must be authenticated by the logon service, win logon.

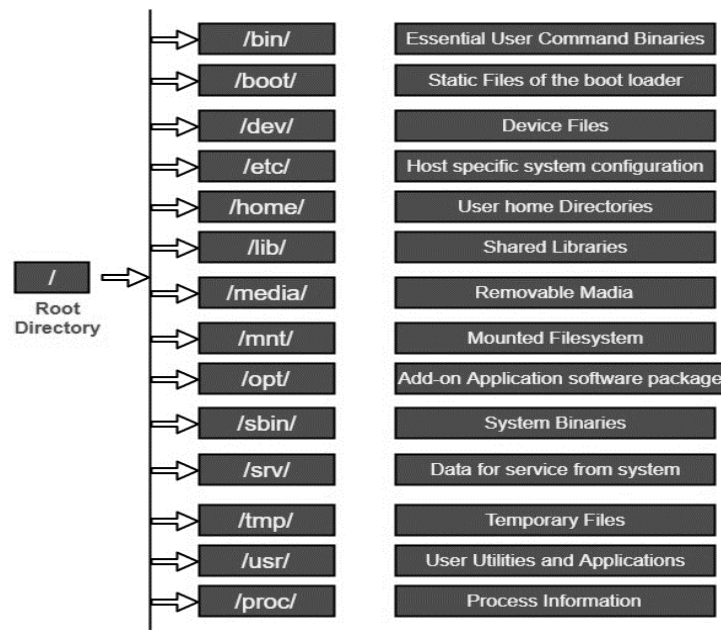
It is responsible for responding to secure attention sequence.

- The local security authority subsystem (LSASS) is the process that generates access tokens to represent users on the subsystems.
- It calls an authentication package to perform authentication using information from the log on subsystem or network server.
- The security subsystem then generates the access token for the user ID containing the appropriate privileges , quota limits and group IDS.
- The default authentication package for windows xp domains is Kerberos.

## 8. Discuss on file system structure in the linux systems. (NOV '15)

The Linux File Hierarchy Structure or the Filesystem Hierarchy Standard (FHS) defines the directory structure and directory contents in Unix-like operating systems. It is maintained by the Linux Foundation.

- In the FHS, all files and directories appear under the root directory /, even if they are stored on different physical or virtual devices.
- Some of these directories only exist on a particular system if certain subsystems, such as the X Window System, are installed.
- Most of these directories exist in all UNIX operating systems and are generally used in much the same way; however, the descriptions here are those used specifically for the FHS, and are not considered authoritative for platforms other than Linux.



## File Systems

Linux retains UNIX's standard file-system model. In UNIX, a file does not have to be an object stored on disk or fetched over a network from a remote file server.

Rather, UNIX files can be anything capable of handling the input or output of a stream of data. Device drivers can appear as files, and interprocess communication channels or network connections also look like files to the user. The Linux kernel handles all these types of file by hiding the implementation details of any single file type behind a layer of software, the virtual file system (VFS).

### The Virtual File System

The Linux VFS is designed around object-oriented principles. It has two components: a set of definitions that specify what file-system objects are allowed to look like and a layer of software to manipulate the objects. The VFS defines four main object types:

- An **inode object** represents an individual file.
- A **file object** represents an open file.
- A **superblock object** represents an entire file system.
- A **dentry object** represents an individual directory entry.

For each of these four object types, the VFS defines a set of operations. Every object of one of these types contains a pointer to a function table. The function table lists the addresses of the actual functions that implement the defined operations for that object.

- `int open (..)` — Open a file.
- `ssize_t read (..)` — Read from a file.
- `ssize_t write (..)` — Write to a file.
- `int mmap (..)` — Memory-map a file.

The complete definition of the file object is specified in the struct `file_operations`, which is located in the file `/usr/include/linux/fs.h`. An implementation of the file object (for a specific file type) is required to implement each function specified in the definition of the file object. The VFS software layer can perform an operation on one of the file-system objects by calling the appropriate function from the object's function table, without having to know in advance exactly what kind of object it is dealing with. The VFS does not know, or care, whether an inode represents a networked file, a disk file, a network socket, or a directory file. The appropriate function for that file's `read` operation will always be at the same place in its function table, and the VFS software layer will call that function without caring how the data are actually read.

The inode and file objects are the mechanisms used to access files. An inode object is a data structure containing pointers to the disk blocks that contain the actual file contents, and a file object represents a point of access to the data in an open file. A process cannot access an inode's contents without first obtaining a file object pointing to the inode. The file object keeps track of where in the file the process is currently reading or writing, to keep track of sequential file I/O.

### The Linux ext2fs File System

The standard on-disk file system used by Linux is called **ext2fs**, for historical reasons. Linux was originally programmed with a Minix-compatible file system, to

ease exchanging data with the Minix development system, but that file system was severely restricted by 14-character file-name limits and a maximum file-system size of 64 MB. The Minix file system was superseded by a new file system, which was christened the **extended file system (extfs)**. A later redesign of this file system to improve performance and scalability and to add a few missing features led to the **second extended file system (ext2fs)**.

It uses a similar mechanism for locating the data blocks belonging to a specific file, storing data-block pointers in indirect blocks throughout the file system with up to three levels of indirection. As in FFS, directory files are stored on disk just like normal files, although their contents are interpreted differently.

Each block in a directory file consists of a linked list of entries; each entry contains the length of the entry, the name of a file, and the inode number of the inode to which that entry refers. The main differences between ext2fs and FFS lie in their disk-allocation policies. In FFS, the disk is allocated to files in blocks of 8 KB. These blocks are subdivided into fragments of 1 KB for storage of small files or partially filled blocks at the ends of files. In contrast, ext2fs does not use fragments at all but performs all its allocations in smaller units. The default block size on ext2fs is 1 KB, although 2-KB and 4-KB blocks are also supported.

To maintain high performance, the operating system must try to perform I/O operations in large chunks whenever possible by clustering physically adjacent I/O requests. Clustering reduces the per-request overhead incurred by device drivers, disks, and disk-controller hardware.

### **Journaling**

Many different types of file systems are available for Linux systems. One popular feature in a file system is **journaling**, whereby modifications to the file system are sequentially written to a journal. A set of operations that performs a specific task is a **transaction**. Once a transaction is written to the journal, it is considered to be committed, and the system call modifying the file system (i.e. write ()) can return to the user process, allowing it to continue execution. Meanwhile, the journal entries relating to the transaction are replayed across the actual file-system structures. As the changes are made, a pointer is updated to indicate which actions have completed and which are still incomplete.

When an entire committed transaction is completed, it is removed from the journal. The journal, which is actually a circular buffer, may be in a separate section of the file system, or it may even be on a separate disk spindle. It is more efficient, but more complex, to have it under separate read-write heads, thereby decreasing head contention and seek times.

If the system crashes, there will be zero or more transactions in the journal. Those transactions were never completed to the file system even though they were committed by the operating system, so they must be completed. The transactions can be executed from the pointer until the work is complete, and the file-system structures remain consistent. The only problem occurs when a transaction has been aborted. That is, it was not committed before the system crashed. Any changes from those transactions that were applied to the file system must be undone, again preserving the

consistency of the file system. This recovery is all that is needed after a crash, eliminating all problems with consistency checking.

### **The Linux proc File System**

The flexibility of the Linux VFS enables us to implement a file system that does not store data persistently at all but rather simply provides an interface to some other functionality. The Linux **process file system**, known as the /proc file system, is an example of a file system whose contents are not actually stored anywhere but are computed on demand according to user file I/O requests.

A /proc file system is not unique to Linux. SVR4 UNIX introduced a /proc file system as an efficient interface to the kernel's process debugging support:

Each subdirectory of the file system corresponded not to a directory on any disk but rather to an active process on the current system. A listing of the file system reveals one directory per process, with the directory name being the ASCII decimal representation of the process's unique process identifier (PID).

Linux implements such a /proc file system but extends it greatly by adding a number of extra directories and text files under the file system's root directory. These new entries correspond to various statistics about the kernel and the associated loaded drivers. The /proc file system provides a way for programs to access this information as plain text files, which the standard UNIX user environment provides powerful tools to process. For example, in the past, the traditional UNIX ps command for listing the states of all running processes has been implemented as a privileged process that reads the process state directly from the kernel's virtual memory. Under Linux, this command

is implemented as an entirely unprivileged program that simply parses and formats the information from /proc. The /proc file system must implement two things: a directory structure and the file contents within.